



React Hook Form & useForm

Cómo gestionar formularios de forma eficiente, accesible y moderna

Aïda García

¿Qué es React Hook Form?

"A performant, flexible and extensible library for managing forms in React."

Librería ligera para formularios

Minimiza re-renders

Validación integrada

API basada en hooks

Accesibilidad más sencilla

Conceptos clave

`useForm()`

`register`

`handleSubmit`

¿Qué es useForm?



useForm es el hook principal de React Hook Form. Proporciona métodos y estado para registrar inputs, validar datos y manejar el envío del formulario.

Devuelve:

`Register()`

Conecta el input con la lógica. Maneja el `onChange`, `onBlur` y la ref automáticamente.

`handleSubmit()`

Valida los campos antes de ejecutar tu función de envío. Olvida el `e.preventDefault()`.

`formState.errors`

Objeto vivo que contiene los errores, si el formulario está "dirty" o si se está enviando.

Inputs No Controlados



A diferencia de `useState`, `useForm` utiliza `Refs`. Esto significa que React **no tiene que redibujar toda la pantalla** cada vez que escribes una letra.

Con `useState`

Escribes "Ana" en el input:

A → React redibuja TODO

An → React redibuja TODO

Ana → React redibuja TODO

= 3 re-renders innecesarios

Con `useForm (Refs)`

Escribes "Ana" en el input:

A → nada, la ref lo guarda

An → nada, la ref lo guarda

Ana → nada, la ref lo guarda

= 0 re-renders hasta enviar

"A diferencia de `useState`, que redibuja la pantalla con cada letra, Hook Form solo actúa cuando le das a enviar o cuando hay un error, ahorrando muchísima batería y CPU en el móvil del usuario."

El formulario en HTML + JavaScript



index.html

```
<form id="miFormulario">

  <div class="campo">
    <label for="nombre">Nombre:</label>
    <input type="text"
      id="nombre"
      required
      placeholder="Ej. Ana" />
  </div>

  <div class="campo">
    <label for="edad">Edad:</label>
    <input type="number"
      id="edad"
      required min="0" max="120"
      placeholder="Ej. 25" />
  </div>

  <button type="submit">Enviar</button>

</form>
```

app.js

```
const form = document
  .querySelector("#miFormulario");

form.addEventListener("submit",
  (e) => {
    e.preventDefault();

    const nombre = document
      .querySelector("#nombre").value;
    const edad = document
      .querySelector("#edad").value;

    console.log({
      nombre,
      edad
    });
  });
```

El mismo formulario con React + useForm



App.jsx

```
import { useForm } from "react-hook-form";

function App() {
  const { register, handleSubmit } = useForm();

  const onSubmit = (data) => {
    console.log(data);
    // { nombre: "Ana", edad: "25" }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <label>Nombre:</label>
      <input {...register("nombre")} />

      <label>Edad:</label>
      <input type="number" {...register("edad")} />

      <button>Enviar</button>
    </form>
  );
}
```

register conecta el input → **handleSubmit** valida y envía → **onSubmit** recibe los datos limpios

Añadiendo validación



App.jsx – con validación

```
const { register, handleSubmit,
  formState: { errors } } = useForm();
<form onSubmit={handleSubmit(onSubmit)}>
  <label>Nombre:</label>
  <input {...register("nombre", {
    required: "El nombre es obligatorio"
  })} />
  {errors.nombre &&
    <p>{errors.nombre.message}</p>}
  <label>Edad:</label>
  <input type="number" {...register("edad", {
    required: "La edad es obligatoria",
    min: { value: 0,
      message: "La edad no puede ser negativa" },
    max: { value: 120,
      message: "Edad máxima: 120" }
  })} />
  {errors.edad &&
    <p>{errors.edad.message}</p>}
  <button>Enviar</button>
</form>
```

¿Por qué usar useForm?



Rapidez

No redibuja toda la pantalla cada vez que escribes. Solo actúa cuando es necesario

Menos código

Con una sola línea conectas un input al formulario. Sin `querySelector` ni `addEventListener`

Errores claros

Los mensajes de error aparecen solos debajo de cada campo si algo está mal

Flexibilidad

Puedes añadir reglas tan simples o complejas como quieras: `required`, `min`, `max`, `pattern`...

Menos errores

Al escribir menos código repetitivo, hay menos sitios donde puedes equivocarte

Preparando el proyecto



1 Crear proyecto con Vite

terminal

```
$ npm create vite@latest  
$ npm install
```

2 Instalar React Hook Form

terminal

```
$ npm install react-hook-form
```

Live Coding



Vamos a crear un formulario paso a paso:

Campos del formulario

- ✓ Nombre
- ✓ Email
- ✓ Botón enviar

Funcionalidades

- ✓ Validación con required
- ✓ Mensajes de error personalizados
- ✓ Mensajes de error visibles
- ✓ `console.log(data)` al enviar

¡Vamos a programar juntas!



¡GRACIAS!

¿Preguntas?